

Yamigisa's 2D Survival Level Editor

Documentation

Table of Contents:

Introduction

This document contains a full documentation of the asset “Yamigisa’s 2D Survival Level Editor”. In here, you will find everything needed to access or use the level editor. If you have unanswered questions, you may contact me at :

Email : ridhodwia4@gmail.com

Discord : yamigisa_

About

Yamigisa's 2D Survival Level Editor focuses on the design and development of a modular and flexible level editor for 2D survival games using the Unity game engine. It aims to address common challenges in game development, such as time inefficiency, technical complexity, and limited access to affordable tools. Core survival mechanics like character movement, environment interaction, inventory, crafting, and data management is structured as a Unity Package, allowing seamless integration into both new and existing projects while minimizing the need to build systems from scratch.

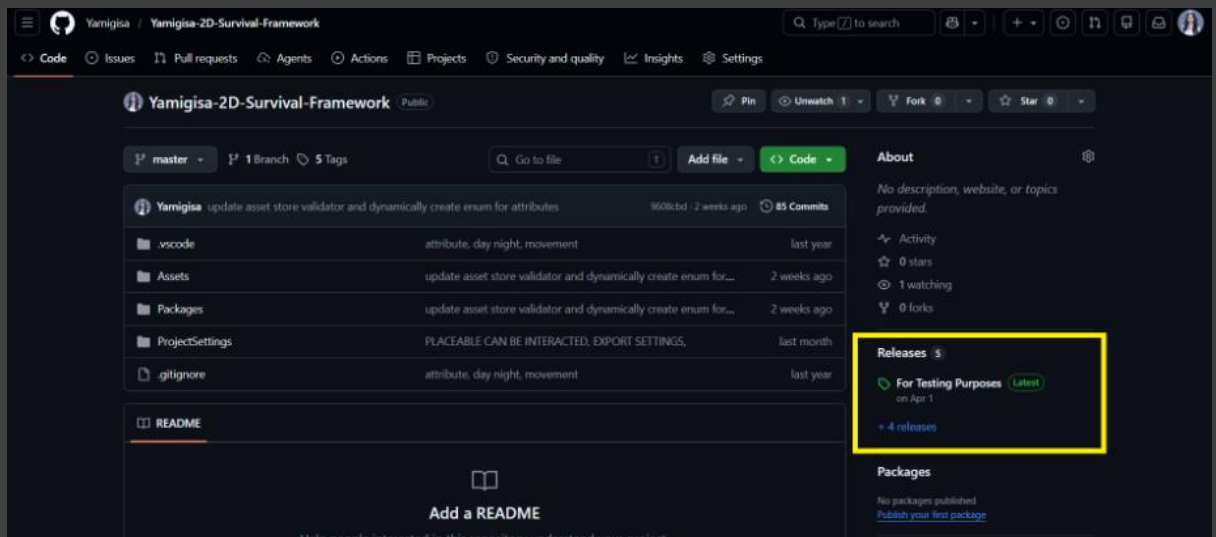
Designed with usability and accessibility in mind, the system emphasizes configuration through Unity's Inspector and custom editor tools rather than direct code modification. This approach enables developers—especially beginners and intermediate users—to rapidly prototype and develop survival games with greater efficiency and consistency. Additionally, the project includes comprehensive documentation to guide users through its structure, features, and implementation workflow, making it not only a development tool but also a learning resource for understanding modular game system design in Unity.

How to Install

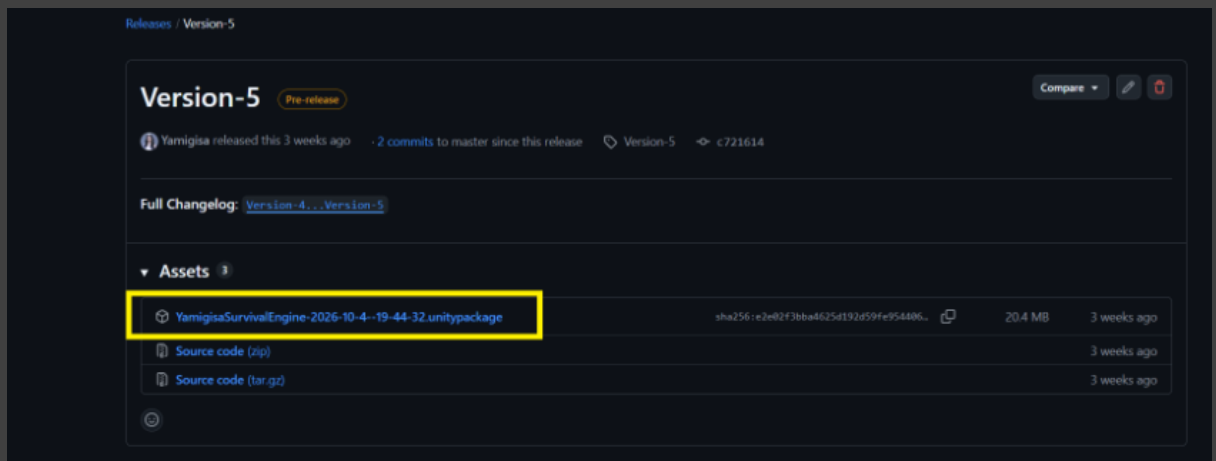
Installation of this project can be done in two different way, using Github or import from Unity Aset Store.

Install from Github:

1. Open the project's github (<https://github.com/Yamigisa/Yamigisa-2D-Survival-Framework>).
2. Open the release section.



3. Click the latest release version.
4. Download the Unity Package File.



5. After download finishes, click on the file and it should automatically open. **If this doesn't work**, create or open a 2D Unity Project and either import or click the package.

Install from Unity Asset Store:

1. Open the project's Unity Asset Store [\(link incoming\)](#).
2. Click “Add to My Assets”.
3. Import project.

Features

Character

The character in this level editor serves as the central entity that connects player input with the game system, acting as the main medium for interaction within the survival environment. It is designed using a modular, component-based structure where features such as movement, input handling, attributes, animation, combat, and inventory are separated into independent but interconnected components. This allows developers to easily customize or extend character functionality without modifying the core structure. The character supports various actions including directional movement, sprinting, jumping, crouching, and interacting with objects or enemies, while also maintaining survival-related attributes like health, hunger, and temperature. This flexible design ensures that the character system can be adapted to different gameplay needs while maintaining efficiency and scalability in development.

Character Features:

1. Movement

The movement system is designed as a flexible, physics-based controller that allows the character to move in a top-down environment using directional input, while supporting additional mechanics such as sprinting, jumping, crouching, and auto-movement. Movement is driven by input from a centralized control system and applied using Rigidbody2D forces to ensure smooth and responsive motion. The system dynamically adjusts speed based on conditions like sprinting, crouching, or external modifiers (e.g., equipment buffs), and restricts certain actions depending on states such as jumping or cooldowns. It also tracks the last movement direction for consistent behavior (e.g., jump direction), and supports automated navigation toward a target position, making it adaptable for both player-controlled and system-driven movement behaviors.

2. Input

The character input system is designed as a unified and flexible input handler that supports both keyboard and gamepad controls through a customizable binding structure. Each action—such as movement, interaction, inventory access, and gameplay actions—is mapped using the InputBinding class, which allows multiple keys and buttons to trigger the same behavior. The system continuously detects

input states (pressed, pressed down, and released) and translates them into gameplay actions through centralized methods, ensuring consistency across different input devices. By separating input logic from gameplay logic, this system enables easy reconfiguration via the Inspector and supports multi-platform input without requiring code changes, making it highly adaptable for various control schemes and devices.

3. **Attribute**

The attribute system represents the core survival mechanics of the character by managing dynamic values such as health, hunger, and thirst through a modular and data-driven approach using ScriptableObjects. Each attribute is defined by its maximum value, current value, and rate of depletion or regeneration over time, which is updated continuously based on in-game events like time progression, environmental (biome) effects, and inter-attribute dependencies. The system also supports advanced interactions where a depleted attribute can influence others through modifiers, creating interconnected survival mechanics. Additionally, attributes are integrated with UI elements that automatically reflect changes in real-time, and can be modified by external systems such as consumables (affecting current values) or equipment (affecting maximum values). This design ensures flexibility, scalability, and efficient data reuse while maintaining clear separation between data, logic, and presentation.

4. **Animation**

The character animation system is a direction-based state controller that dynamically updates animations based on the character's movement state and last input direction. It works by reading data from the movement system—specifically whether the character is currently walking and the last movement direction—and translating that into corresponding animation states such as idle or walking in front, back, or side directions. The system uses boolean parameters in the Animator to control transitions, ensuring only one animation state is active at a time. Additionally, it handles sprite flipping for left and right movement by modifying the transform scale, allowing a single side animation to represent both directions. This approach keeps the animation logic simple, efficient, and tightly synchronized with gameplay behavior without requiring complex animation state machines.

5. **Combat**

The character combat system is designed as an automated, target-based attack mechanism that integrates closely with movement and damage calculation systems. When a target is assigned, the character will continuously evaluate its distance to the target and automatically move within attack range if necessary. Once in range, the system executes attacks at fixed intervals using a cooldown timer, ensuring consistent and controlled damage output. The damage calculation is modular, prioritizing equipment-based damage when available and falling back to base (hand) damage otherwise, with additional modifiers applied through buffs. The system also intelligently cancels attacks when the player manually interrupts movement or when the target becomes invalid, making combat responsive and adaptable to player actions while maintaining automation for continuous engagement.

Environment

The environment system serves as the main gameplay space where all game activities take place, functioning not only as a visual setting but also as an active system that influences gameplay. It includes key elements such as the time cycle (day and night), interactive objects (like trees, rocks, and resources), and special areas such as biomes that can affect the character's condition. Each element is designed in a modular way using prefabs and ScriptableObjects, allowing developers to easily configure and expand the system without modifying core code. The environment is also integrated with other systems, such as attributes and interactions, where environmental conditions can impact the character's status or trigger specific behaviors, creating a dynamic and responsive gameplay experience.

Environment Features

1. Time Cycle

The time cycle system manages the progression of in-game time by continuously updating minutes, hours, and days at a configurable real-time interval, allowing the game world to feel alive and responsive. It not only tracks time progression, but also triggers events whenever the minute, hour, or day changes, enabling other systems such as attributes, environment logic, or gameplay events to react accordingly. In addition, the system updates UI elements like the clock and day display, controls a visual clock fill, and adjusts day-night lighting through a gradient-based lighting system using Light2D. Because it is implemented as a

centralized singleton and supports save/load integration, the time cycle acts as a core world system that keeps time, visuals, and dependent gameplay systems synchronized consistently throughout the game.

2. Interactive Object

The interaction system is divided into two main types: `InteractiveObject` and `Destroyable`, where every interactable object in the game must have an `InteractiveObject`, but only some objects additionally implement `Destroyable`. The `InteractiveObject` acts as the core interaction layer, handling player input, action execution, hover detection, UI feedback, and optional regrowth behavior for resources. It defines how the player interacts with objects through configurable actions and ensures interaction is consistent across all objects. On the other hand, `Destroyable` extends this system by adding health, damage handling, loot dropping, and death behavior, making it suitable for objects that can be attacked and destroyed (such as trees, rocks, or enemies). This separation allows objects to remain lightweight and flexible—simple objects can be interacted with without destruction logic, while more complex objects can include full combat and lifecycle behavior without duplicating interaction logic.

3. Biome

The biome system defines environmental regions that shape both the visual identity and gameplay behavior of an area. Each biome is stored as a `'ScriptableObject'` and contains data such as its name, ground representation (either a tile or a prefab), spawn tables for resources and creatures, and environmental modifiers that affect the character. When the player enters a biome area through a `'BiomeTrigger'`, the system applies that biome to the character using `'CharacterBiome'`, which then updates movement speed and attribute-related effects such as regeneration or depletion changes. This makes biomes more than just visual zones, since they directly influence how the player survives and interacts with the world.

The world generation system uses chunks to build the game world in a modular and scalable way. Each `'WorldChunk'` is responsible for generating its own terrain, either through a tilemap or a biome prefab, and for spawning biome-specific resources and enemies within its bounds. The `'WorldGenerator'` manages which chunks should exist, spawning them around the player based on a configurable load radius and assigning biome groups according to chunk coordinates. This

allows the world to expand dynamically as the player moves, while also supporting editor-based generation, object spawning, refreshing, and save/load integration. Together, the biome and world generation systems create a structured procedural world where each area has its own identity, content, and gameplay effects.

Creatures

The wild creature and enemy system is a data-driven AI behavior system that controls how non-player creatures move, detect the player, react to threats, and perform attacks. Each creature uses `AnimalData` as its configuration source, which defines movement values, detection settings, combat properties, and behavior type such as passive, escaping when attacked, defending when attacked, or always aggressive. At runtime, the `Animal` component manages wandering within a defined range, checking the player's position and vision angle, switching into escape or attack behavior when appropriate, and handling animation states based on movement direction and current action. This system is also integrated with `Destroyable`, allowing creatures to take damage, play death animations, drop loot, and notify the destruction system when death is complete. Because the logic is modular and behavior is controlled by data, the same system can be reused for harmless animals, defensive creatures, or hostile enemies with minimal changes.

Item

The item system is a data-driven system that defines all usable, collectible, and interactable objects in the game through a centralized `ItemData ScriptableObject`. Each item contains complete information such as visuals, type (resource, equipment, consumable, or placeable), stack rules, actions, and gameplay effects like consumable buffs or equipment stats. The `Item` component acts as a lightweight runtime representation that links a world object to its corresponding `ItemData`, while the `ItemDatabase` provides a global lookup system to retrieve items efficiently using unique IDs. This design allows items to be easily reused, extended, and managed without modifying code, while also supporting advanced systems such as crafting, regrowth, equipment bonuses, and attribute effects in a modular and scalable way.

Inventory

The inventory system is a comprehensive item management system that handles storing, organizing, using, and transferring items between different contexts such as quick slots, main inventory, equipment, and storage. It is designed with a dual-panel structure (main

inventory and quick access slots) and supports features such as stacking, sorting, drag-and-drop interaction, crafting, and equipment integration. The system prioritizes usability by allowing players to quickly access items through hotkeys or scroll selection, while also enabling advanced interactions like splitting stacks, equipping items, and consuming items through action-based logic. Additionally, the inventory dynamically adjusts its capacity based on equipped items (e.g., bags), supports saving/loading, and communicates changes through events, making it a central hub that connects item, UI, crafting, and character systems into a unified workflow.

Crafting

The crafting system is a modular and data-driven feature that allows players to create new items by consuming required materials or item groups defined in each ItemData. It is tightly integrated with the inventory system, where crafting availability is dynamically checked based on the player's current items, and UI elements automatically update to reflect whether a recipe can be crafted. The system is organized into craft groups that categorize recipes, supports unlocking additional crafting categories at runtime, and provides a structured interface for selecting recipes, viewing requirements, and executing crafting actions. By separating crafting logic, UI, and data definitions, the system ensures flexibility, scalability, and ease of expansion for adding new recipes or crafting mechanics without modifying core code.

Placeable

The placeable system allows players to construct and place objects dynamically in the game world using either grid-based or free placement modes. Each placeable object is represented by the Placeable component, which defines its occupied area, placement state, and visual feedback during placement. The system is managed by PlaceableSystem, which handles entering build mode, previewing placement using tilemaps (in grid mode), validating placement through collision and area checks, and finalizing placement by consuming the corresponding inventory item. It also supports real-time visual feedback (valid/invalid placement colors), prevents overlapping objects, and integrates with inventory and interaction systems to ensure smooth building gameplay. This modular design allows developers to easily configure building behavior, grid constraints, and placement rules while maintaining flexibility for different gameplay styles.

Data

The data system is responsible for preserving the game state by collecting, storing, and restoring information from all major gameplay systems through a centralized save architecture. SaveManager acts as the main controller that creates a SaveGameData container, asks every object implementing ISavable to write its state into that container, and then serializes the result into a JSON save file. The system supports manual saving and loading, automatic saving over time, optional auto-save when the player sleeps, and selective save toggles for different subsystems such as world time, player data, inventory, destroyables, interactive objects, chunks, storages, and equipment. Because each gameplay system saves only its own data structure while sharing the same save pipeline, the design remains modular, scalable, and easy to extend with new saveable components.

Create Object Editor

The create object system is an editor-based tool designed to streamline the creation of game assets such as items, destroyables, animals, placeables, biomes, and characters through a unified workflow. It uses CreateObjectSettings as a centralized configuration that defines folder paths, default actions, and global rules applied during object creation, ensuring consistency across all generated assets. Through the custom CreateObjectWindow, developers can generate both ScriptableObject data (e.g., ItemData, AnimalData) and corresponding prefabs in a single step, with automatic assignment of components, layers, colliders, visuals, and default interaction behaviors. This system significantly reduces manual setup, enforces standardized structure, and accelerates content creation by combining data and prefab generation into a single automated pipeline.

Duplicate Object Editor

The duplicate object system is an editor utility designed to quickly create new items by cloning existing ItemData assets while optionally generating a new prefab and modifying visual or data properties. It streamlines content creation by allowing developers to reuse existing item configurations, reducing repetitive setup while maintaining consistency across assets. Through the DuplicateObject wizard, users can choose whether to retain the original item's data, override icons, or create a completely new prefab that mirrors the structure of objects created by the main creation tool. This system ensures efficient iteration, safe asset duplication, and organized file management

by automatically handling folder creation, naming validation, and asset linking between data and prefab.

Replace Prefab in Scene Editor

The replace prefab in scene system is an editor utility designed to efficiently swap all instances of a specific prefab in the active scene with another prefab while preserving their transform properties and hierarchy structure. It scans the entire scene hierarchy, identifies objects that are instances of the selected source prefab, and replaces them with the new prefab using Unity's prefab system. During replacement, it maintains position, rotation, scale, parent, and sibling index to ensure the scene layout remains unchanged. The system also integrates with Unity's Undo system, allowing safe batch operations and easy rollback, making it highly useful for rapid iteration, asset updates, or refactoring large scenes without manually replacing objects one by one.

Scene Initializer Editor

The scene initializer system is an editor utility designed to streamline the process of setting up a new scene by automatically creating and organizing essential objects based on predefined settings. It helps generate required scene components such as managers, player objects, cameras, UI elements, lighting, or other configured prefabs without needing to place them manually one by one. The system uses initialization settings to determine which objects should be created, where they should be placed, and how they should be organized in the hierarchy, allowing each scene to follow a consistent structure. This makes the scene creation process faster, reduces setup mistakes, and improves workflow efficiency, especially when working with multiple scenes that require the same core systems or object configuration.

Create New Scene Editor

The create new scene system is an editor utility designed to efficiently generate a new Unity scene while automatically initializing the required objects defined in the scene settings. It allows the user to input a scene name, choose an optional target folder, or use the default folder from the initialization settings when no folder is selected. When creating the scene, the system validates the scene name, ensures the destination folder exists, prevents duplicate scene files, and saves the new scene as a Unity scene asset. After the scene is created, it scans the configured required object list and instantiates each prefab into the scene while avoiding duplicates based on object names. The system also supports Unity's prefab workflow and integrates with the Undo system for safer

editor operations, making it useful for quickly creating consistent scene structures without manually setting up core objects every time.

Scripting API

Character

Movement API (CharacterMovement.cs)

Method / Property	Explanation
Move()	Handles manual player movement based on input (WASD / controller). Applies velocity using Rigidbody2D.
AutoMove()	Moves the character automatically toward a target position. Used for scripted movement or AI-like behavior.
MoveTo(Vector2 target, float stoppingDistance)	Starts auto-movement toward a specific position with a stopping threshold.
StopAutoMoveExternal()	Public method to stop auto-movement from external scripts.
DisableAllMovements()	Disables all movement-related actions, including move, sprint, jump, and crouch.
EnableAllMovements()	Re-enables all movement-related actions.
HandleSprint()	Manages sprint logic, including stamina usage, cooldown, and regeneration.
HandleJump()	Controls jump behavior, including arc movement and landing logic.

Method / Property	Explanation
HandleCrouch()	Handles crouch state, including collider disabling and sprite scaling.
SetSpeedMultiplier(float addition)	Adds an additional speed modifier, such as buffs or effects.
AddSpeedBuff(float amount)	Increases the total speed buff and updates movement speed.
RemoveSpeedBuff(float amount)	Reduces the speed buff and recalculates movement speed.
SetEquipmentMoveSpeedBonus(float flat, float percent)	Applies movement bonuses from equipped items using flat and percentage values.
IsWalking	Returns whether the character is currently moving.
LastDirection	Stores the last movement direction, which is used for jump and facing direction.
IsAutoMoving	Indicates whether the character is currently auto-moving.

Input Scripting API (CharacterInput.cs)

Method / Property	Description
IsPressed(InputBinding binding)	Returns true while the input is being held as continuous input. Supports both keyboard and gamepad.
IsPressedDown(InputBinding binding)	Returns true only on the frame when the input is first pressed as a single trigger.
IsPressedUp(InputBinding binding)	Returns true on the frame when the input is released.

Method / Property	Description
GetMouseCameraRay()	Converts the mouse position into a world ray using the main camera. Useful for clicking or interaction systems.
gamepad	Stores the currently active gamepad device using Gamepad.current.
moveUp / moveDown / moveLeft / moveRight	Input bindings for directional movement.
sprint	Input binding for the sprint action.
jump	Input binding for the jump action.
crouch	Input binding for the crouch action.
interaction1 / interaction2	Input bindings for interacting with objects using primary or secondary interaction.
inventory	Input binding for opening the inventory UI.
useItem	Input binding for using items, such as mouse click or trigger input.
crafting	Input binding for opening the crafting system.
cancel	Input binding for canceling actions or closing UI.
pause	Input binding for pausing the game.
Reset()	Initializes the default keyboard and gamepad bindings for all actions.

Attribute Scripting API

1. Core Data (AttributeData.cs)

Method / Property	Description
type	Defines the attribute type (Health, Hunger, Thirst).
MaxValue	Maximum value of the attribute.
CurrentValue	Current value of the attribute.
BaseMaxValue	Original max value (used for recalculation with equipment).
DepleteValuePerMinute	Amount reduced every minute (set 0 to disable).
RegenerateValuePerMinute	Amount regenerated every minute (set 0 to disable).
triggerGameOver	Triggers game over when value reaches 0.
DepletedModifiers	Effects applied to other attributes when this one is depleted.
MoveSpeedLossValue	Movement penalty when attribute is depleted.
GetAttributeType()	Returns the attribute type.

2. Attribute Logic (CharacterAttribute.cs)

Method / Property	Description
type	Defines the attribute type (Health, Hunger, Thirst).
MaxValue	Maximum value of the attribute.
CurrentValue	Current value of the attribute.
BaseMaxValue	Original max value (used for recalculation with equipment).
DepleteValuePerMinute	Amount reduced every minute (set 0 to disable).
RegenerateValuePerMinute	Amount regenerated every minute (set 0 to disable).

Method / Property	Description
triggerGameOver	Triggers game over when value reaches 0.
DepletedModifiers	Effects applied to other attributes when this one is depleted.
MoveSpeedLossValue	Movement penalty when attribute is depleted.
GetAttributeType()	Returns the attribute type.

3. UI System (AttributeUI.cs & AttributeBar.cs)

Method / Property	Description
InitializeAttributeBar(AttributeData)	Creates or updates UI bar for an attribute.
GetAttributeBar(AttributeData)	Retrieves the UI bar of a specific attribute.
SetAttributeBar(AttributeData)	Applies attribute data to UI visuals (fill, background, values).

4. Supporting Behaviour

Feature	Description
Depleted Attribute Effects	Attributes at 0 can affect others (e.g., hunger increases health drain).
Biome Modifiers	Environment dynamically affects regen/deplete rates.
Equipment System	Modifies max values only (not current).
Consumables	Modify only current values.
UI Sync	Automatically updates sliders in real-time.

Animation Scripting API

Method / Property	Description
Start()	Initializes required components such as Animator, CharacterMovement, and Transform.
Update()	Main animation handler that updates animation states every frame based on movement data.
animator	Reference to Unity Animator component used to control animation states.
movement	Reference to CharacterMovement to get direction and movement state.
spriteTransform	Reference to transform used for flipping sprite (left/right).
idleFront / idleBack / idleSide	Animator parameter names for idle states based on direction.
walkFront / walkBack / walkSide	Animator parameter names for walking states based on direction.
movement.LastDirection	Provides the last movement direction used to determine facing direction.
movement.IsWalking	Indicates whether the character is currently moving.

Combat Scripting API

1. Core Methods

Method / Property	Description
StartAutoAttack(Destroyable target)	Starts attacking a target automatically and resets cooldown.

Method / Property	Description
StopAttack()	Stops attacking, clears target, and stops auto-movement.
Update()	Main combat loop: handles targeting, movement to target, cooldown, and damage application.
IsAttacking (Property)	Returns whether the character is currently attacking.

2. Damage System

Method / Property	Description
CalculateBaseDamage()	Determines base damage (equipment damage prioritized, fallback to hand damage).
GetFinalDamage(int baseDamage)	Applies buffs to base damage and returns final damage value.
handDamage	Default damage when no equipment is used.
equipmentDamage	Damage value provided by equipped items.
damageBuff	Additional bonus damage applied on top of base damage.

3. Attack Behaviour

Feature	Description
Auto Targeting	Attacks a selected Destroyable target continuously.
Range Checking	Uses squared distance to determine if target is within attack range.
Auto Movement	Moves character toward target if out of range.
Cooldown System	Controls attack rate using attackCooldown.

Feature	Description
Interrupt Handling	Stops attack when player moves manually or target becomes null.

4. Buff & Equipment API

Method	Description
AddDamageBuff(float amount)	Increases total damage output.
RemoveDamageBuff(float amount)	Decreases damage bonus.
SetEquipmentDamage(int value)	Sets damage value from equipped weapon.

Time Cycle Scripting API

Method / Property	Description
Instance	Static singleton reference to the active TimeClock system.
Minute	Current in-game minute value.
Hour	Current in-game hour value.
Day	Current in-game day value.
OnMinuteChanged	Event invoked whenever the minute changes.
OnHourChanged	Event invoked whenever the hour changes.
OnDayChanged	Event invoked whenever the day changes.
Setup()	Initializes the time system, wires events, applies default starting values, and refreshes visuals.
StartSystem()	Starts the ticking coroutine so time begins progressing.
StopSystem()	Stops the ticking coroutine and pauses time progression.

Method / Property	Description
SetTime(int minute, int hour, int day)	Manually sets the current minute, hour, and day values.
ForceRefreshVisual()	Forces the UI and lighting to update immediately based on the current time.
Save(ref SaveGameData data)	Saves the current day, hour, and minute into save data.
Load(SaveGameData data)	Loads saved time values, refreshes visuals, and restarts the system.

InteractiveObject Scripting API

1. Core Interaction

Method / Property	Description
Actions	List of ActionBase that defines what interactions can be performed.
InteractObject(Character character)	Executes the default interaction (first action).
IsCharacterInRange(Character character)	Checks if character is within interaction range.
interactRange	Maximum distance for interaction.

2. Input & Hover System

Method	Description
HandleMouseHover()	Detects hovered object and shows outline + tooltip.
HandleMouseClicked()	Handles input-based interaction (interaction1 / interaction2).
SetOutline(bool on)	Enables or disables visual outline.

3. Regrowth System (Interactive Object)

Method	Description
CanStartRegrowth()	Checks if object supports regrowth (resource type).
StartRegrowth()	Starts regrowth process after harvesting.
HandleHarvested()	Called after object is harvested/destroyed.
ApplyCurrentGrowthStage()	Applies sprite and timing of current growth stage.
AdvanceGrowthStage()	Moves to next growth stage.
FinishRegrowth()	Restores object to original state after regrowth.

Biome Scripting API

Method / Field	Description
biome	The biome assigned to this chunk.
groundTilemap	Tilemap used when generating tile-based terrain.
size	Chunk size setting used during initialization.
seed	Random seed used for procedural generation within the chunk.
resourcesSpawned	Indicates whether resources have already been spawned.
enemiesSpawned	Indicates whether enemies have already been spawned.
Initialize(BiomeData biome, int size, int seed)	Initializes the chunk and starts building it.

Method / Field	Description
Initialize(BiomeData biome, int size, int seed, bool spawnObjectsOnBuild)	Initializes the chunk with optional object spawning during generation.
ForceImmediateGeneration()	Forces the chunk to generate terrain and objects immediately.
GenerateObjectsOnlyImmediate()	Generates only resources and enemies without rebuilding terrain.
SetSpawnFlags(bool resources, bool enemies)	Sets whether resource and enemy spawning are considered completed.
ClearSpawnedObjects()	Removes all spawned objects from the chunk.
GetWorldSize()	Returns the chunk's world-space dimensions.

Creatures Scripting API

Method / Property / Field	Description
animalData	Reference to the AnimalData asset used by this creature.
Start()	Initializes movement, animation hashes, vision cache, and wander state.
Update()	Main AI loop that handles detection, wandering, escaping, attacking, and reaction timing.
FixedUpdate()	Ensures the creature stops properly when dead or attacking, and updates idle animation when necessary.

Method / Property / Field	Description
OnEnable()	Subscribes to Destroyable.OnKilled and caches the current HP.
OnDisable()	Unsubscribes from Destroyable.OnKilled.
DetectCharacter(float range)	Checks whether the player is within range and inside the configured vision angle.
ReactDetected()	Decides what action to take after the player is detected based on AnimalBehaviour.
EscapeFromCharacter()	Moves the creature away from the player using run speed.
AttackCharacter()	Handles chasing, attack timing, cooldown, and damage application.
Wander()	Controls random wandering movement and temporary idle pauses.
PickNewWanderTarget()	Chooses a new random position inside the wander range.
FaceDirection(Vector2 dir)	Updates the last facing direction used for animation.
SetAnimation(AnimMode mode, Vector2 dir)	Switches animation booleans based on the current action and direction.
SafeSetBool(string param, bool value)	Safely sets an animator boolean only if the parameter exists.
OnAttacked()	Marks the creature as attacked so its behavior can change accordingly.
OnDestroyableKilled(Destroyable d)	Triggered when the linked Destroyable dies, then stops AI and starts the death animation flow.

Method / Property / Field	Description
DeathAnimThenLoot(Destroyable d)	Waits for the death animation to finish, then notifies the Destroyable system.
destroyable	Reference to the Destroyable component used for HP, death, and loot integration.
isDead	Indicates whether the creature has already died.
isAttacked	Indicates whether the creature has been damaged by the player.
isAttacking	Indicates whether the creature is currently performing an attack.
attackCooldownTimer	Stores the remaining cooldown time before the next attack can start.
reactionTimer	Tracks how long the creature has been detecting the player before reacting.
isDetectedLocked	Indicates whether the player has been fully detected and the reaction has started.
wanderCenter	Center point used as the origin of the creature's wandering area.
wanderTarget	Current destination during wandering.
lastAnimDir	Last direction used for facing and animation selection.
GlobalFreeze	Global flag used to freeze all animal behavior.

Item Scripting API

Field / Method	Description
Id	Unique identifier for the item, used for saving, loading, and item lookup.
itemName	Display name of the item shown to the player.
description	Text description of the item shown in the UI.
itemPrefab	Prefab used when spawning the item in the world.
iconWorld	Sprite used when the item appears in the world.
iconInventory	Sprite used for the item inside the inventory UI.
itemType	Defines the item category, such as Resource, Equipment, Consumable, or Placeable.
maxAmount	Maximum amount of the item that can be stacked.
isDroppable	Determines whether the item can be dropped from the inventory.
isStackable	Determines whether multiple items can be stacked in one slot.
groups	Defines item groups used for categorization or crafting requirements.
itemActions	List of actions that can be performed with the item, such as use, drop, or split.
isCraftable	Determines whether the item can be crafted.
craftGroupsNeeded	Required item groups needed to craft this item.
craftItemsNeeded	Specific items required to craft this item.

Field / Method	Description
craftResultAmount	Amount of items produced after crafting.
ChangeDroppableState(bool state)	Changes whether the item can be dropped or not.
Get(string id)	Finds and returns an item data asset based on its ID.

Inventory Scripting API

Method / Property	Description
Instance	Singleton instance of the inventory system.
Setup()	Initializes inventory slots, inventory panels, quick slots, and starting items.
ShowInventory()	Opens the inventory UI and locks character input.
HideInventory()	Closes the inventory UI and restores gameplay input.
IsInventoryOpen	Returns whether the inventory UI is currently open.
AddItem(ItemData data, int amount, InventoryPanel panel = null)	Adds an item to the inventory, prioritizing quick slots first.
ReduceSlotAmount(ItemSlot slot, int amount)	Reduces the item amount in a slot or clears it when the amount reaches zero.
CanAddItem(ItemData data, int amount)	Checks whether the inventory has enough space for the item.
UseSlot(ItemSlot slot)	Executes valid actions from the selected item slot.

Method / Property	Description
TryPlaceSplit(ItemData data, int amount)	Places a split item stack into an available slot.
GetSelectedQuickSlot()	Returns the currently selected quick slot.
GetSelectedQuickItemData()	Returns the item data from the selected quick slot.
SelectQuickSlot(int index)	Selects a quick slot based on its index.
UseQuickSlot(int index)	Uses the item stored in the selected quick slot.
SortInventory()	Sorts the main inventory items by type and name.
BeginDrag(ItemSlot origin)	Starts dragging an item from a slot.
CompleteDragAtCursor()	Completes the drag action by swapping, equipping, or dropping the item.
CancelDrag()	Cancels the current drag operation.
GetUsedSlotCount()	Returns the number of currently occupied inventory slots.
GetCurrentCapacity()	Returns the current inventory capacity, including bag bonuses.
SetEquipmentBagBonus(int value)	Updates inventory capacity based on equipped bag bonuses.
Save(ref SaveGameData data)	Saves the current inventory state.
Load(SaveGameData data)	Restores inventory data from a save file.

Crafting Scripting API

Method / Property	Description
Setup()	Initializes the crafting system, crafting groups, UI, and inventory event connections.
OpenCraftingItemSelectionPanel(CraftGroupSlot slot)	Opens the crafting item selection UI based on the selected crafting group.
CloseAllCraftingInterfaces()	Closes all crafting panels and resets the current crafting state.
OpenCraftingFromPlaceable(GroupData group)	Opens the crafting UI from an external object, such as a crafting station.
BuildCraftSelection()	Loads available crafting groups and creates the crafting group UI slots.
AddItemsForGroup(GroupData group)	Displays all craftable items that belong to the selected crafting group.
AddAdditionalCraftGroup(GroupData group)	Adds and unlocks a new crafting group during gameplay.
ForceInitialize()	Ensures the crafting system is initialized before being used.

Method / Property	Description
TryCraft(ItemData recipe)	Attempts to craft an item if the player has the required materials.
UpdateCraftButtonState()	Updates whether the craft button can be pressed based on crafting requirements.
OnInventoryChanged()	Refreshes the crafting UI when the inventory content changes.
ClearItemList()	Clears the current list of craftable items from the crafting UI.
SelectFirstCraftItem()	Automatically selects the first available craftable item.
RefreshAllItemSlotsInteractable()	Updates the interactable state of all crafting item buttons.

Placeable Scripting API

Method / Property	Description
Placed	Indicates whether the object has already been placed in the world.
area	Defines the grid size or area occupied by the object.
placedArea	Stores the final occupied area after the object is placed.

Method / Property	Description
Place()	Finalizes object placement, snaps it to the grid if needed, and registers the occupied area.
OnDestroy()	Releases the occupied area when the placed object is destroyed.
SetSpriteColor(bool canPlace)	Changes the object color to show whether the placement is valid or invalid.
OnValidate()	Ensures the placement area setup is valid in the editor.
OnDrawGizmos()	Draws placement area visualization in the editor.
instance	Singleton reference to the placement system.
useGrid	Determines whether placement uses grid mode or free placement mode.
IsInBuildMode	Returns whether the player is currently in build mode.
IsPlacingObject	Indicates whether an object is currently being placed.
IsInteractionBlocked	Prevents interaction shortly after placing an object.
EnterBuildMode()	Enables build mode and prepares the placement preview.
ExitBuildMode()	Disables build mode and clears temporary preview tiles.

Method / Property	Description
InitializeBuilding(GameObject prefab)	Starts placing a new object from a prefab.
InitializeBuilding(GameObject prefab, ItemSlot sourceSlot)	Starts placing a new object and links it to an inventory item slot.
CancelBuild()	Cancels the current placement process and removes the preview object.
CanTakeArea(BoundsInt area)	Checks whether the selected area is valid for placement.
TakeArea(BoundsInt area)	Marks an area as occupied after successful placement.
ReleaseArea(BoundsInt area)	Frees an occupied area when an object is removed.
GetCellCenterWorld(BoundsInt area)	Returns the world position at the center of a grid area.
RebuildGrid(Vector3Int anchorCell)	Updates the buildable grid area around the player.
FollowBuilding()	Updates the preview object, preview tiles, and placement validation.
ClearArea()	Clears temporary placement preview tiles.
HasBlockingCollider(BoundsInt area)	Checks whether a grid placement area overlaps with blocking colliders.
HasBlockingColliderWorld(Placeable object)	Checks whether a free placement object overlaps with blocking colliders.

Method / Property	Description
ConsumeBuildItemIfNeeded()	Removes the related inventory item after successful placement.

Data Scripting API

Method / Property	Description
Instance	Singleton reference to the active save manager.
SaveGame()	Collects data from all savable objects and writes the data into the save file.
LoadGame()	Reads the save file, reconstructs the saved data, and restores it to all savable objects.
TriggerSleepAutoSave()	Triggers automatic saving when sleep-based auto-save is enabled.
Awake()	Initializes the save manager singleton.
Update()	Handles manual save/load input and interval-based auto-save.
OnApplicationQuit()	Saves the game automatically when the application closes, if enabled.
saveGameOnQuit	Determines whether the game should save automatically when quitting.
enableAutoSave	Enables periodic automatic saving.
autoSaveInterval	Sets the time interval between automatic saves.
autoSaveOnSleep	Enables automatic saving when the player sleeps.
path	Stores the save file location inside the persistent data path.
SaveWorldTime	Determines whether world time data should be saved.

Method / Property	Description
SavePlayer	Determines whether player data should be saved.
SaveInventory	Determines whether inventory data should be saved.
SaveDestroyables	Determines whether destroyable object data should be saved.
SaveInteractiveObjects	Determines whether interactive object data should be saved.
SaveChunks	Determines whether world chunk data should be saved.
SaveStorages	Determines whether storage data should be saved.
SaveEquipment	Determines whether equipped item data should be saved.
Save(ref SaveGameData data)	Writes an object's data into the shared save container.
Load(SaveGameData data)	Reads saved data from the shared save container and restores the object state.

Create Object Editor Scripting API

Method / Property	Description
prefabFolder	Base folder used to store generated prefabs.
prefabItemsFolder	Folder used to store item-related prefabs.
itemsFolder	Resources folder used to store ItemData assets.
prefabAnimalsFolder	Folder used to store animal prefabs.

Method / Property	Description
animalDataFolder	Resources folder used to store AnimalData assets.
interactiveObjectLayer	Layer assigned to created interactive objects.
defaultItemActions	Default actions applied to created items.
defaultConsumableActions	Default actions applied to consumable items.
defaultEquipmentActions	Default actions applied to equipment items.
defaultInteractiveObjectActions	Default actions applied to interactive objects.
defaultDestroyableActions	Default actions applied to destroyable objects.
defaultAnimalActions	Default actions applied to animal objects.
globalPlaceableItemDataActions	Actions automatically added to all placeable item data.
defaultPlaceableActions	Default actions assigned based on the placeable object type.
Open()	Opens the Create Object editor window.
OnGUI()	Displays the editor interface and handles object creation input.
AutoFindSettings()	Automatically searches for the CreateObjectSettings asset.
CreateItemOrDestroyable()	Creates item or destroyable data and prefab assets.

Method / Property	Description
CreatePlaceable()	Creates a placeable item and prefab with placement functionality.
CreateAnimal()	Creates an animal prefab and its AnimalData asset.
CreateBiome()	Creates a biome trigger prefab.
CreateCharacter()	Creates a character prefab with the required character systems.
EnsureFolder(string path)	Creates the required folder path if it does not exist.
MakeSafeFileName(string name)	Converts a name into a safe file name for asset creation.
EnsureItemTypeFolderExists(ItemType type)	Ensures the folder for a specific item type exists.
GetItemDataFolderForType(ItemType type)	Returns the correct ItemData folder based on the item type.

Duplicate Object Editor Scripting API

Method / Property	Description
source	Original ItemData asset that will be duplicated.
duplicatedName	Name used for the new duplicated item.
makeNewPrefab	Determines whether a new prefab should also be created.

Method / Property	Description
iconInventory	Optional inventory icon used for the duplicated item.
iconWorld	Optional world icon used for the duplicated item.
retainItemInfo	Determines whether the duplicated item keeps the original item data or resets it.
Open()	Opens the Duplicate Object editor wizard.
OnWizardUpdate()	Updates the wizard help text based on the current input.
DrawWizardGUI()	Displays the input fields and options in the wizard window.
OnWizardCreate()	Runs the duplication process for the item data and optional prefab.
Validate Input	Checks whether the source item and duplicated name are valid.
Ensure Folders	Creates the required folders if they do not exist.

Method / Property	Description
Copy or Create Data	Duplicates the original ItemData or creates a new empty item data asset.
Apply Modifications	Updates the duplicated item name, icons, and related settings.
Create Prefab	Creates a new prefab for the duplicated item when enabled.
Save & Refresh	Saves the generated assets and refreshes the Unity Editor.
CreateItemPrefabLikeCreateObject(ItemData so, string path, string name)	Creates an item prefab with collider, visual, outline, and interactive setup based on the item data.

Replace Prefab In Scene Editor Scripting API

Method / Property	Description
sourcePrefab	Prefab asset that will be searched and replaced in the active scene.
newPrefab	Prefab asset used as the replacement object.
Open()	Opens the Replace Prefab editor window.
OnGUI()	Displays the editor UI for selecting prefabs and starting the replacement process.
ReplaceInActiveScene()	Finds all instances of the source prefab in the active scene and replaces them with the new prefab.

Method / Property	Description
IsPrefabAsset(GameObject go)	Checks whether the selected object is a valid prefab asset from the Project window.
Validate Prefabs	Ensures the source and replacement objects are valid prefab assets.
Traverse Hierarchy	Searches through all objects in the active scene hierarchy.
Detect Prefab Instances	Checks whether a scene object is an instance of the selected source prefab.
Preserve Transform	Keeps the original position, rotation, scale, parent, and hierarchy order.
Replace Object	Instantiates the new prefab and removes the old object.
Mark Scene Dirty	Marks the scene as modified so the changes can be saved.